

VyaparFlow

Frontend and Backend Design Transcript

React/Next.js Frontend with FastAPI Backend for an LLM-Enabled Business Operating System

This transcript explains the application-facing design of VyaparFlow: frontend screens, UI state, backend FastAPI structure, API contracts, authentication, data flow, and integration points for future Agentic AI and RAG pipelines.

Frontend	React or Next.js with TypeScript, Tailwind CSS, TanStack Query, Zustand/Context
Backend	FastAPI with Python, Pydantic schemas, SQLAlchemy ORM, Alembic migrations
Database	PostgreSQL for transactional business data
Cache/Jobs	Redis for sessions, caching, command state, and background queues
AI Readiness	Python backend directly supports LLM SDKs, speech processing, embeddings, and RAG
Next Document	Agentic AI and RAG pipeline workflow design

1. Opening Context

Presenter:

In this design transcript, I will explain the frontend and backend structure of VyaparFlow. Since VyaparFlow depends on voice commands, LLM-based intent understanding, and later Agentic AI and RAG workflows, the backend is better implemented using FastAPI in Python. FastAPI gives us strong API performance, async support, automatic documentation, Pydantic validation, and direct access to the Python AI ecosystem.

Presenter:

The frontend will focus on a simple voice-first user experience, while the backend will handle authentication, business records, command processing, database transactions, and integration points for AI services. The goal is to keep the UI simple and the backend modular enough to connect agents and RAG cleanly in the next phase.

2. Final Recommended Tech Stack

Layer	Technology	Purpose
Frontend Framework	React or Next.js with TypeScript	Responsive web dashboard and voice-first interface.
Styling	Tailwind CSS or shadcn/ui	Fast, consistent, accessible UI components.
Client State	Zustand or React Context	Local UI state such as active command, recorder state, filters.
Server State	TanStack Query	API fetching, caching, retries, optimistic updates.
Backend API	FastAPI	Async Python API layer for business and AI workflows.
Validation	Pydantic	Request/response schemas and type-safe validation.
ORM	SQLAlchemy 2.0	Database models, sessions, transactions, queries.
Migrations	Alembic	Versioned PostgreSQL schema changes.
Database	PostgreSQL	Products, sales, expenses, customers, commands, audit logs.
Cache/Queue	Redis + Celery/RQ/Arq	Background processing, command state, retries, notifications.
Object Storage	S3-compatible storage	Audio files, invoices, report exports.
AI Services	Speech-to-text, LLM, embeddings, text-to-speech	Voice and conversational intelligence.

3. Frontend Responsibilities

- Provide a voice-first command input with record, pause, retry, and submit states.
- Show dashboard summaries for sales, stock, expenses, dues, and insights.
- Let users view and manage products, inventory, transactions, customers, reports, and reminders.
- Display command status while the backend processes speech, LLM parsing, and workflow execution.
- Ask for confirmation when the backend detects ambiguity or risky actions.
- Support multilingual text display and future text-to-speech responses.
- Keep the UI simple enough for small business owners who may not be comfortable with complex software.

4. Frontend App Structure

```
apps/web/
src/
  app/ or pages/
    dashboard/
    products/
    transactions/
    customers/
    insights/
    settings/
  components/
    voice/
      VoiceRecorder.tsx
      CommandStatusPanel.tsx
      ConfirmationPrompt.tsx
    dashboard/
      MetricCard.tsx
      SalesChart.tsx
      LowStockList.tsx
  hooks/
    useVoiceRecorder.ts
    useCommandStatus.ts
    useBusinessContext.ts
  services/
    apiClient.ts
    commandApi.ts
    productApi.ts
    transactionApi.ts
  store/
    authStore.ts
    businessStore.ts
    uiStore.ts
```

Presenter:

The frontend is organized around features and reusable UI components. The voice folder contains the most important interaction components, such as the recorder, command status panel, and confirmation prompt. Services contain typed API calls, while hooks and stores manage UI state and server state.

5. Core Frontend Screens

Screen	Purpose	Important UI Elements
Login/Onboarding	Authenticate user and create/select business.	OTP/password input, language choice, business profile form.
Dashboard	Daily operational summary.	Voice command bar, sales cards, low-stock alerts, dues, insights.
Voice Command View	Show current command processing state.	Transcript, detected intent, progress steps, confirmation request.
Products/Inventory	Manage stock and product catalog.	Product table, stock adjustment, reorder level, category filter.
Transactions	View sales, purchases, and payments.	Transaction list, filters, payment status, export button.
Customers	Track customer dues and purchase history.	Customer list, balance due, last purchase, follow-up reminders.
Insights	Show AI-generated business suggestions.	Pricing suggestions, demand trends, weekly report, confidence labels.
Settings	Business preferences and privacy settings.	Language, currency, audio retention, negative stock policy.

6. Frontend State Design

- Authentication state stores access token, user profile, selected business, and role.
- Voice recorder state tracks idle, recording, uploading, processing, needs_confirmation, completed, and failed states.
- Server state is fetched with TanStack Query for dashboard, products, transactions, customers, and insights.

- Optimistic updates can be used for simple UI changes, but financial and inventory writes should wait for backend confirmation.
- Command polling or WebSocket/SSE can be used to update long-running command status.

7. Backend Responsibilities

Presenter:

The FastAPI backend is responsible for API routing, validation, authentication, business logic, persistence, and AI integration. It should not be a thin database wrapper. It must enforce business rules such as tenant isolation, stock validation, payment status validation, and idempotent command processing.

- Expose REST APIs for frontend screens and voice command workflows.
- Validate all requests using Pydantic schemas.
- Use SQLAlchemy models and repositories for database access.
- Use service classes for business logic and transactions.
- Handle voice command lifecycle from input to result.
- Provide integration boundaries for LLM, speech-to-text, embeddings, and text-to-speech.
- Run background jobs for analytics refresh, reports, reminders, and future RAG indexing.
- Write audit logs for all business-critical and agent-assisted actions.

8. FastAPI Backend Folder Structure

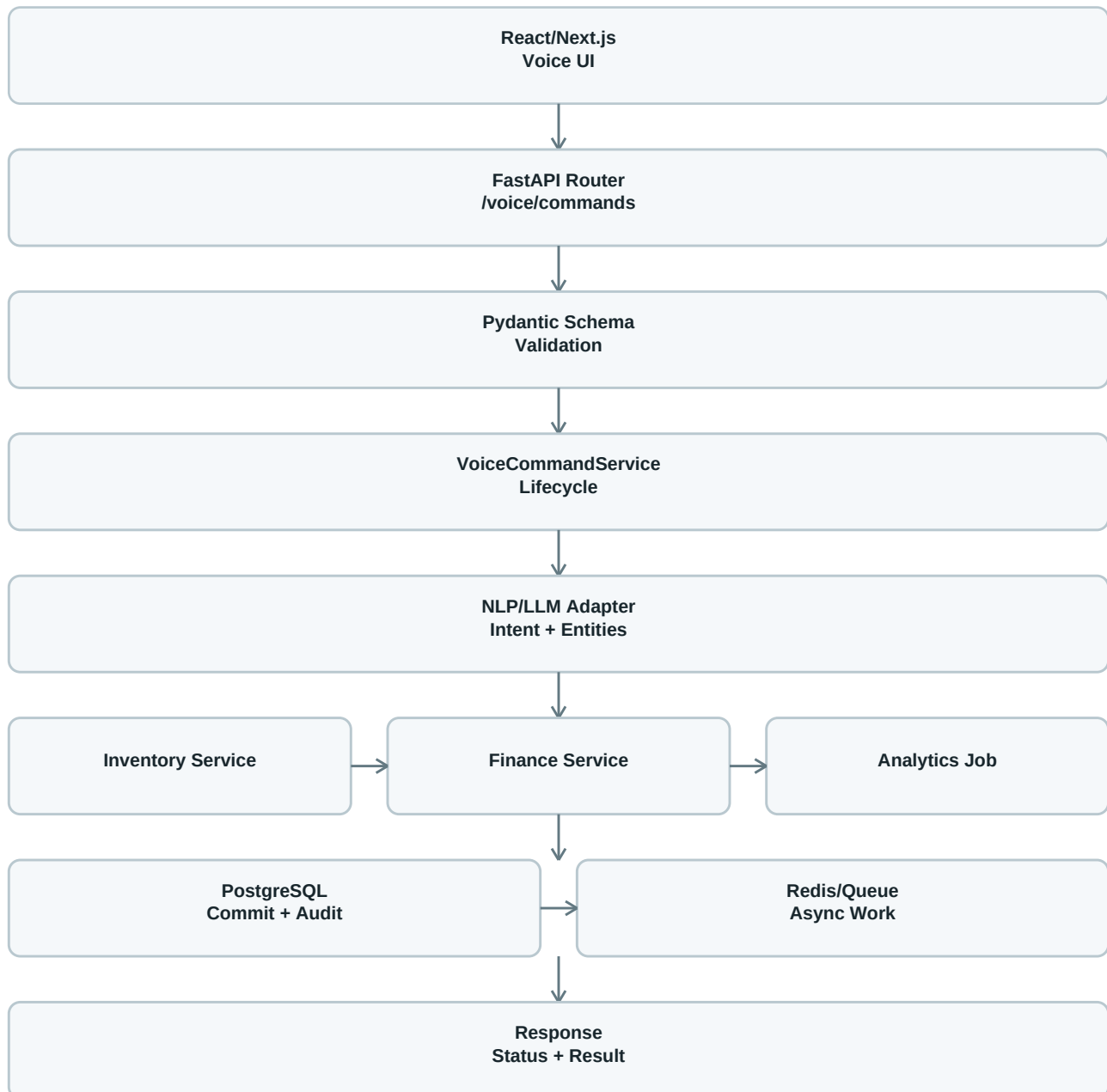
```
apps/api/  
  app/  
    main.py  
    core/  
      config.py  
      security.py  
      database.py  
      redis.py  
  api/  
    v1/  
      router.py  
      endpoints/  
        auth.py  
        businesses.py  
        voice_commands.py  
        products.py  
        transactions.py  
        analytics.py  
  models/  
    user.py  
    business.py  
    product.py  
    transaction.py  
    voice_command.py  
    agent_task.py  
    audit_log.py  
  schemas/  
    auth.py  
    product.py  
    transaction.py  
    voice_command.py  
    analytics.py  
  services/  
    auth_service.py  
    voice_command_service.py  
    nlp_service.py  
    inventory_service.py  
    finance_service.py  
    analytics_service.py  
  repositories/  
    product_repository.py  
    transaction_repository.py  
    command_repository.py  
  integrations/  
    speech_client.py  
    llm_client.py  
    storage_client.py  
  workers/  
    jobs.py  
  tests/
```

9. Backend Module Design

Module	FastAPI Layer	Responsibility
Auth	auth.py, auth_service.py	Login, JWT, roles, selected business context.
Business	businesses.py, business_service.py	Business profile, settings, membership.
Voice Commands	voice_commands.py, voice_command_service.py	Create command, upload audio, track status, return result.
NLP	nlp_service.py, speech_client.py, llm_client.py	Transcript, language detection, intent/entity extraction.
Inventory	products.py, inventory_service.py	Product CRUD, stock adjustments, low-stock validation.
Finance	transactions.py, finance_service.py	Sales, expenses, transaction items, payment status.
Analytics	analytics.py, analytics_service.py	Dashboard summaries, metrics, trend snapshots.
Insights	insights.py	Business suggestions and advisor outputs.
Notifications	reminders.py, notification_service.py	Reminders, alerts, future WhatsApp/SMS hooks.
Audit	audit_log.py	Append-only logs for user, system, and future agent actions.

10. Request Flow Diagram

Frontend to FastAPI Request Flow



11. API Design Overview

Endpoint	Method	Used By Frontend For
/api/v1/auth/login	POST	User login or OTP verification.
/api/v1/businesses	GET/POST	Business selection and setup.
/api/v1/voice-commands	POST	Submit audio or text command.
/api/v1/voice-commands/{id}	GET	Poll command status and result.
/api/v1/voice-commands/{id}/confirm	POST	Confirm ambiguous or risky action.
/api/v1/products	GET/POST	Product list and creation.
/api/v1/products/{id}/stock	PATCH	Manual stock adjustment.
/api/v1/transactions	GET/POST	Sales, expenses, and transaction history.
/api/v1/customers	GET/POST	Customer records and dues.
/api/v1/analytics/dashboard	GET	Dashboard metrics.
/api/v1/insights	GET	AI recommendations and reports.

12. Pydantic Schema Examples

```
class VoiceCommandCreate(BaseModel):
    input_type: Literal["audio", "text"]
    text: str | None = None
    audio_file_id: UUID | None = None
    business_id: UUID
    idempotency_key: str

class ParsedCommand(BaseModel):
    command_id: UUID
    language: str
    intent: Literal["SALE", "EXPENSE", "STOCK_UPDATE", "REPORT", "UNKNOWN"]
    entities: list[dict]
    confidence: float

class SaleCreate(BaseModel):
    product_id: UUID
    quantity: Decimal
    unit_price: Decimal | None = None
    customer_id: UUID | None = None
    payment_status: Literal["PAID", "DUE", "PARTIAL"]
```

13. FastAPI Service Pattern

```
@router.post("/voice-commands")
async def create_voice_command(
    payload: VoiceCommandCreate,
    user: CurrentUser,
    session: AsyncSession = Depends(get_session),
):
    return await voice_command_service.create_and_process(
        payload=payload,
        user=user,
        session=session,
    )

class VoiceCommandService:
    async def create_and_process(self, payload, user, session):
        command = await self.command_repo.create(payload, user, session)
        parsed = await self.nlp_service.parse(command)
        plan = await self.workflow_planner.plan(parsed)
        result = await self.workflow_executor.execute(plan, session)
        return result
```

The endpoint stays thin. Validation happens through Pydantic, and business logic sits inside service classes. This keeps testing clean and makes it easier to plug in the later Agentic AI workflow executor.

14. Database Access Design

- Use SQLAlchemy 2.0 async sessions for database access.
- Use Alembic for database migrations.
- Keep models in app/models and Pydantic schemas in app/schemas.
- Use repositories for query-heavy operations and services for business behavior.
- Wrap sale workflows in transactions so inventory and finance remain consistent.
- Use business_id in every business-owned table for tenant isolation.
- Use idempotency_key for voice commands to prevent duplicate transactions.

15. Frontend Backend Integration

Frontend Action	Backend API	Result
User records command	POST /voice-commands	Command ID and initial status.
UI shows processing	GET /voice-commands/{id}	Status: transcribing, planning, executing, completed, failed.
Backend needs clarification	GET command status	Frontend displays confirmation prompt.
User confirms product/price	POST /voice-commands/{id}/confirm	Workflow resumes.
Dashboard opens	GET /analytics/dashboard	Sales, expenses, low stock, insights.
Product page opens	GET /products	Product catalog and stock levels.
Manual stock update	PATCH /products/{id}/stock	New stock level and stock movement.

16. Voice Command UI States

State	Frontend Behavior	Backend Status
idle	Show microphone button.	No active command.
recording	Show timer, stop button, audio waveform.	No backend request yet.
uploading	Disable submit and show progress.	Audio/text is being sent.
transcribing	Show transcript loading.	Speech service running.
understanding	Show intent detection message.	LLM/NLP parsing command.
executing	Show workflow steps.	Domain services updating records.
needs_confirmation	Show confirmation prompt.	Backend waiting for user input.
completed	Show success and updated dashboard.	Workflow committed.
failed	Show retry/correction option.	Workflow failed or rolled back.

17. Authentication and Security

- Use JWT access tokens with refresh token support.
- Add business_id and role to the authenticated request context.
- Protect every business API with tenant checks.
- Use HTTPS in deployment and secure cookies if browser-based sessions are used.
- Never expose raw LLM prompts, provider keys, or internal tool payloads to the frontend.
- Require confirmation for risky actions such as negative stock, deletion, or invoice sending.
- Mask sensitive data in logs and audit LLM-related operations carefully.

18. Background Jobs

- Use Redis with Celery, RQ, or Arq for async jobs.
- Run analytics refresh after sales, expenses, and stock movements.
- Generate invoices and reports outside the main request when possible.
- Process reminders and low-stock alerts asynchronously.
- Prepare for future RAG indexing jobs when documents, invoices, or business memory are added.
- Retry temporary failures with bounded retry counts and dead-letter logging.

19. Why FastAPI Is Better Here

- The project will use LLMs, speech processing, embeddings, and RAG; Python has the strongest ecosystem for these workflows.
- FastAPI supports async APIs, which helps with speech services, LLM calls, and long-running command processing.
- Pydantic gives strict validation for AI-produced structured outputs before they reach business services.
- FastAPI automatically provides OpenAPI documentation, useful for frontend integration.
- The same backend language can serve normal APIs and AI orchestration code without maintaining a separate AI microservice initially.

Short version: React handles the user experience, while FastAPI handles business APIs and AI-native backend workflows.

20. Testing Strategy

- Frontend component tests for recorder, command status panel, forms, and dashboard widgets.
- Frontend integration tests for command submission and confirmation flows.
- FastAPI unit tests for services, validators, repositories, and schemas.
- API tests using pytest and httpx AsyncClient.
- Database tests for transaction rollback, idempotency, tenant isolation, and stock consistency.
- Mocked AI provider tests for speech-to-text and LLM parsing responses.
- End-to-end tests for sale, expense, stock update, and clarification workflows.

21. Closing Summary

Presenter:

To summarize, the frontend should be built with React or Next.js and should focus on a clean voice-first experience. The backend should use FastAPI because VyaparFlow is AI-heavy and will later need Agentic AI and RAG workflows. FastAPI gives us strong validation, async support, clean API design, and direct access to Python LLM tooling.

Presenter:

The frontend and backend should communicate through clear REST APIs. The frontend should never directly manage business-critical logic. The FastAPI backend should validate requests, enforce tenant isolation, execute business transactions, and expose safe integration points for LLMs, speech processing, agents, and retrieval pipelines.

Takeaway: use React/Next.js for the voice-first business UI and FastAPI for a Python-native, LLM-ready backend that can grow into Agentic AI and RAG workflows.

Next planned document: Agentic AI and RAG pipeline workflow for VyaparFlow.